

Ex 1:

we wanna optimize the l2 distance, with slot count = 4 under the preference vector [0.2, 0.8]
(w_keys = 0.8)

We have 2 lists of optimization steps:

List 1:

- rot-add-vectorize (0)
- rot-add-vectorize (0)
- rot-mul-vectorize (0)
- sub-vectorize (0)
- END

The resulted expression: exec_cost = 261, keys_cost = 3 {1, 2, 4}

List 2:

- rot-add-vectorize (0)
- add-vectorize (0)
- mul-vectorize (0)
- mul-vectorize (0)
- sub-vectorize (0)
- sub-vectorize (0)
- sub-vectorize (0)
- sub-vectorize (0)
- END

The resulted expression: exec_cost = 262, keys_cost = 1 {1}

We calculate the rewards for those 2 preference vectors:

	[1, 0]	[0.2, 0.8]
List 1	0.9304	-0.294
List 2	0.9302	0.026

- **If we use the unified approach:** we will choose directly **Expression 2**
- **If we use the separated approach:**
 - first we will optimize only on execution time, so we choose **Expression 1**
 - We pass this expression to the next agent and we optimize for the vector [0.2, 0.8]
 - the 2nd agent has 3 options:
 - return the expression directly (END): **reward = 0**
 - decompose the rotation of 4 into 2 rotations of 2 (keys count reduces from 3 to 2): **reward = 0.306**
 - decompose all rotations into a rotation of 1 (use only one key): **reward = 0.377**
 - Going with option 3, the resulting expression will have: **exec_cost = 465, keys_cost = 1 {1}**, which makes Expression 2 chosen by the unified approach better than it.

Ex 2: (END action)

For high w_{keys} values (for ex 0.95), the **unified agent** will tend to end the optimization directly and return the scalar code. The **separated approach** will always tend to optimize execution time first.